

SOCKET OPTIONS

Byeong-hee Roh

Dept. of Software and Computer Engineering

Ajou University



변화와 융합의 중심

MMcN

멀티미디어 융합 연구실

Mobile Multimedia convergene Network Lab.

<http://mmcn.ajou.ac.kr>



Contents

- ❖ Socket Options
- ❖ UDP Broadcast Socket Programming
- ❖ UDP Multicast Socket Programming



SOCKET OPTIONS



Socket Options

❖ Socket options

- can change socket operation mode for executing applications more appropriately
- socket options are defined at each level
 - at socket level
 - `SOL_SOCKET`
 - at transport level
 - `SOL_TCP` (or `IPPROTO_TCP`), `SOL_UDP` (or `IPPROTO_UDP`)
 - at IP level
 - `SOL_IP` (or `IPPROTO_IP`),
 - `SOL_IPX`, `SOL_AX25`, `SOL_ATALK`, etc

❖ Two functions for socket options

- `getsockopt( )` – to retrieve a socket option being used
- `setsockopt( )` – to set a option on a socket

Socket Options

❖ setsockopt()/getsockopt() – set/get socket options

```
int setsockopt
    (int          s,
     int          level,
     int          optname,
     const void   *optval,
     socklen_t    optlen);
```

```
int getsockopt
    (int          s,
     int          level,
     int          optname,
     const void   *optval,
     socklen_t    *optlen);
```

■ arguments

- `s` : socket descriptor
- `level` : the protocol level of the option
- `optname` : the option name to set
- `optval` : the pointer of a buffer for the new option value
- `optlen` : the length of the buffer pointed to by the `optval` parameter in bytes

■ return values

- `0`, when successful
- `-1`: when an error occurs

Socket Options

❖ SOL_SOCKET : Generic Socket Options

level	optname	get	set	type	value	description
SOL_SOCKET	SO_DEBUG	○	○	BOOL	0 or 1	it enables socket debugging
	SO_REUSEADDR	○	○	BOOL	0 or 1	it enables local address reuse
	SO_KEEPALIVE	○	○	BOOL	0 or 1	it enables keep TCP connections alive by sending keepalive message
	SO_LINGER	○	○	struct linger{ }	time	it blocks close() call returns until a defined time is expired
	SO_BROADCAST	○	○	BOOL	0 or 1	it enables permission to transmit broadcast messages
	SO_SNDBUF	○	○	int	bytes	it can set/get the buffer size (in bytes) for output
	SO_RCVBUF	○	○	int	bytes	it can set/get the buffer size (in bytes) for input
	SO_TYPE	○	X	int	1	it can get the type of the socket, e.g. SOCK_STREAM, SOCK_DGRAM, etc. (get only)
	SO_RCVLOWAT	○	○	int	bytes	it defines minimum count (in bytes) of input data to return recv() function
	SO_SNDLOWAT	○	○	int	bytes	it defines minimum count (in bytes) for output
	SO_OOBINLINE	○	○	BOOL	0 or 1	it enables reception of out-of-band data in band

ping
get

Socket Options

❖ IPPROTO_TCP : TCP Socket Options

level	optname	get	set	type	value	description
IPPROTO_TCP	TCP_MAXRT	O	O	int	seconds	it defines the <u>maximum retransmission</u> time of TCP
	TCP_MAXSEG	O	O	int	bytes	it defines the <u>MSS</u> (maximum segment size) of TCP
	TCP_NODELAY	O	O	BOOL	0 or 1	it disables Nagle's algorithm.
	TCP_KEEPINTVL	O	O	int	bytes	it sets interval between consecutive keepalive messages
	TCP_INFO	O	X	struct tcp_info		it is used to collect information about this socket

Socket Options

❖ IPPROTO_UDP : UDP Socket Options

level	optname	get	set	type	value	description
IPPROTO_UDP	UDP_SEGMENT	0	0	int	bytes	it enables UDP segmentation offload.
	UDP_GRO	0	0	int	bytes	it enables UDP receive offload, is the inverse of segmentation offload.
	UDP_NOCHECKSUM	0	0	BOOL	0 or 1	when TRUE, UDP datagrams are sent with the checksum of zero. (winsock only with ws2tcpip.h)

Socket Options

❖ IPPROTO_IP : IP Socket Options

level	optname	get	set	type	value	description
IPPROTO_IP	IP_TTL	○	○	int	0-255	it sets time-to-live (TTL) of unicast IP packets
	IP_TOS	○	○	int		it sets Type of Service (TOS) settings
	IP_MTU	○	×	int		it gets the system's estimate of the path MTU.
	IP_OPTIONS	○	○	char		it specifies IP options to be inserted into outgoing packets.
	IP_MULTICAST_TTL			uchar	0-255	it sets time-to-live of multicast IP packets
	IP_MULTICAST_LOOP			int	0 or 1	it enables to receive its sending multicast packets
	IP_ADD_MEMBERSHIP			ip_mreq { }	group address	it set the group address to receive multicast packets
	IP_DROP_MEMBERSHIP			ip_mreq { }	group address	it drops the group address to receive multicast packets



Socket Options

❖ More socket options

■ for linux

- for SOL_SOCKET
 - <https://man7.org/linux/man-pages/man7/socket.7.html>
- for IPPROTO_TCP
 - <https://man7.org/linux/man-pages/man7/tcp.7.html>
- for IPPROTO_UDP
 - <https://man7.org/linux/man-pages/man7/udp.7.html>
- for IPPROTO_IP
 - <https://man7.org/linux/man-pages/man7/ip.7.html>
- for IPPROTO_RAW
 - <https://man7.org/linux/man-pages/man7/raw.7.html>

■ for windows

- <https://learn.microsoft.com/windows/win32/winsock/socket-options>

Socket Option Examples: Socket Buffers

❖ Buffer Related Options

- `SO_SNDBUF`
- `SO_RCVBUF`

❖ Examples

- To get buffer sizes

```
int bufsize;  
optlen = sizeof bufsize;  
  
z = getsockopt (s, SOL_SOCKET, SO_SNDBUF, &bufsize, &optlen);    // send buffer  
z = getsockopt (s, SOL_SOCKET, SO_RCVBUF, &bufsize, &optlen);    // receive buffer
```

- to set buffer sizes

```
bufsize = 5000;  
z = setsockopt (s, SOL_SOCKET, SO_SNDBUF, &bufsize, optlen);    // send buffer  
z = setsockopt (s, SOL_SOCKET, SO_RCVBUF, &bufsize, optlen);    // receive buffer
```

Socket Option Examples: SO_TYPE

❖ SO_TYPE

- it can only be retrieved.
- it tells what kind of socket it is dealing with

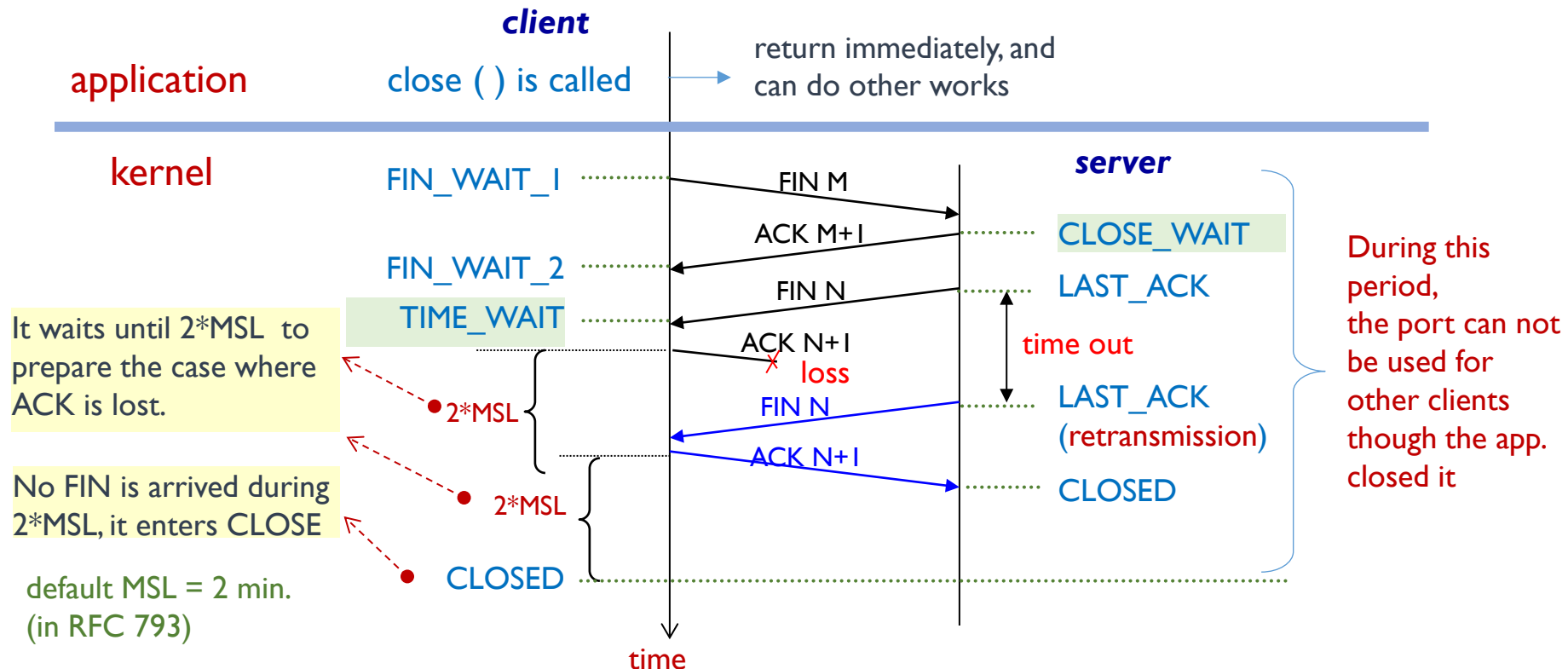
```
int so_type, optlen;  
  
optlen = sizeof(so_type);  
  
z = getsockopt (s, SOL_SOCKET, SO_TYPE, &so_type, &optlen);  
  
printf("the socket type is %d\n",so_type);
```



```
// linux/net.h  
enum sock_type {  
    SOCK_STREAM    = 1,  
    SOCK_DGRAM     = 2,  
    SOCK_RAW       = 3,  
    ...  
};
```

Socket Option Examples: SO_LINGER (1/2)

- ❖ Default behavior for the `close()`
 - for UDP, the kernel closes the socket immediately
 - for TCP, default TCP closing connection process works
 - unnecessary long `TIME_WAIT` problem



Socket Option Examples: SO_LINGER (2/2)

❖ SO_LINGER

- it controls how the socket is shut down when the function `close( )` is called.
 - the option applies only to connection-oriented protocol (e.g. TCP)
- the structure `linger`

```
struct linger {  
    int    l_onoff;    // for linger, it set to TRUE  
    int    l_linger;   // the timeout period (in second) for close( )  
}
```

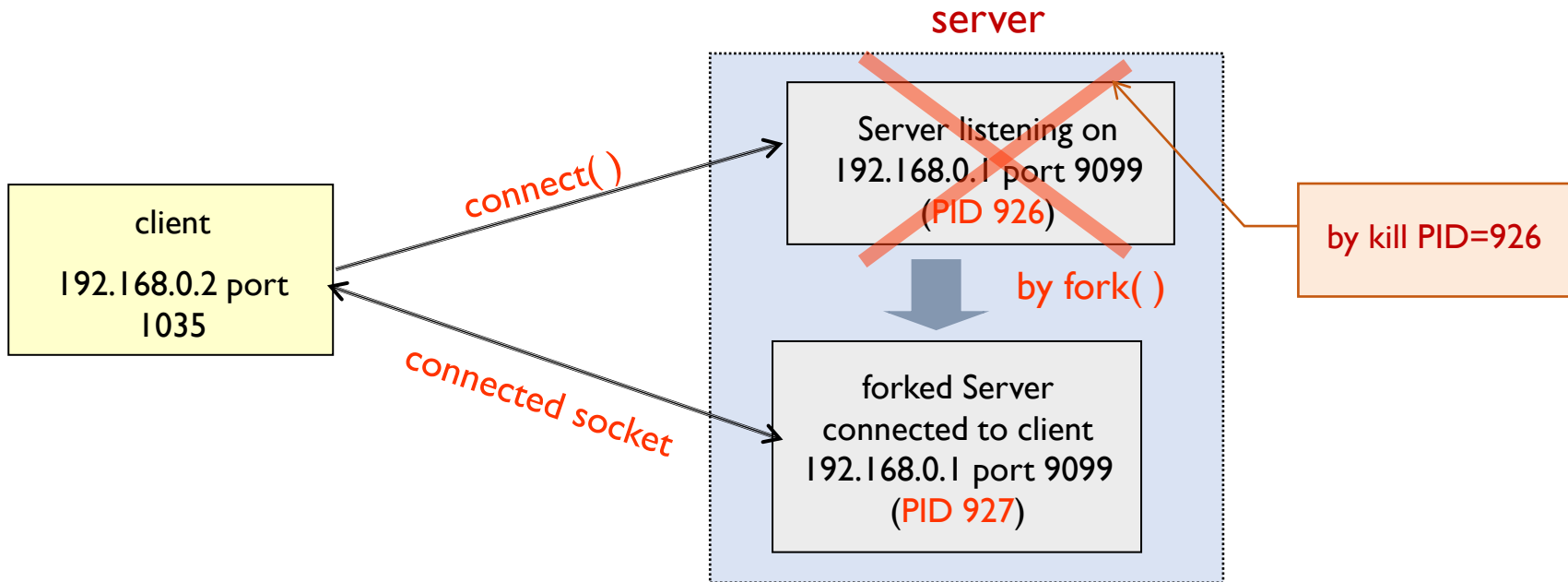
linger 구조 설명
1 -) TRUE 동기종료
0 -) 비동기종료
대기시간

- example – to shutdown the socket close in 10 sec

```
struct linger so_linger;  
...  
so_linger.l_onoff = TRUE;  
so_linger.l_linger = 10;           // in seconds  
  
z = setsockopt ( s, SOL_SOCKET, SO_LINGER, &so_linger, sizeof so_linger );
```

Socket Option Examples: SO_REUSEADDR (1/4)

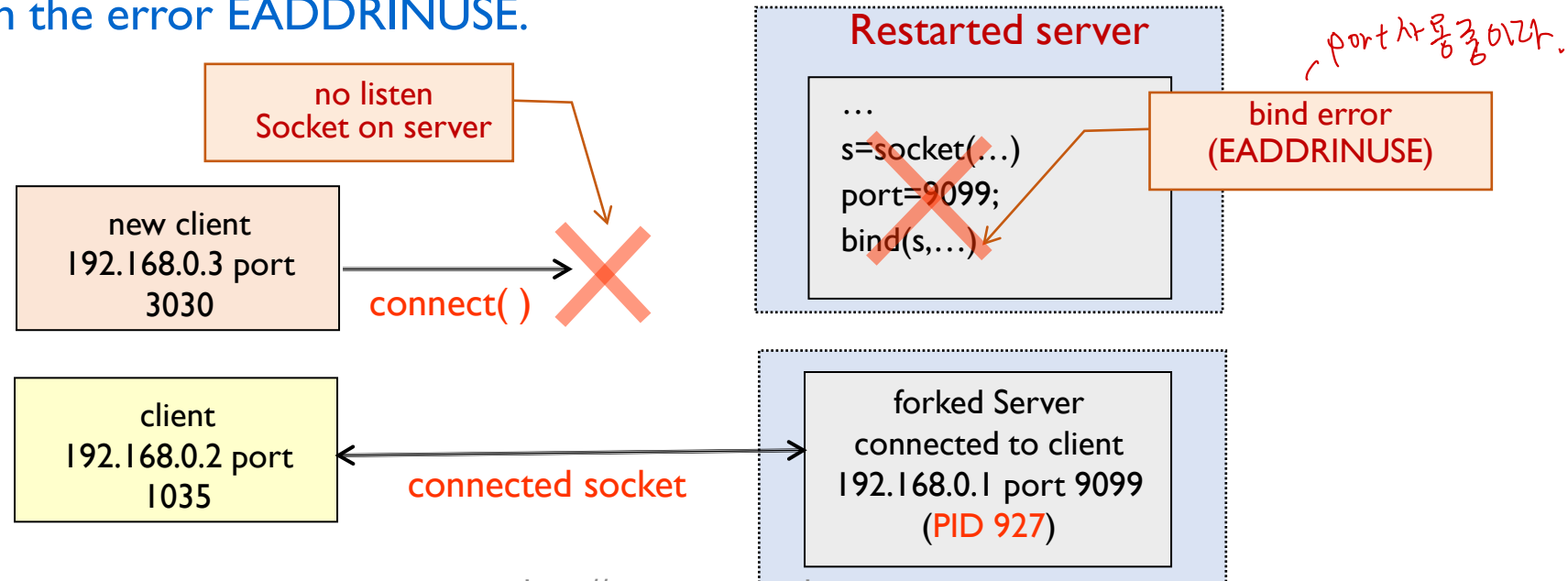
- ❖ Example – need for SO_REUSEADDR option (1/2)
 - Server (parent) (PID= 926) creates clone for a client
 - The client is being serviced by the clone process (PID=927)
 - though PID 926 is killed, the client will continue to be serviced.



Socket Option Examples: SO_REUSEADDR (2/4)

❖ Example – need for SO_REUSEADDR option (2/2)

- no new connections to the server can be made
 - because there will be no server listening for new connections (listening server PID 926 was killed)
- restarting the server has a problem
 - when it attempts to bind the IP address 192.168.0.1 port 9099, the bind() function will return the error EADDRINUSE.



Socket Option Examples: SO_REUSEADDR (3/4)

❖ example use of SO_REUSEADDR option

```
int s ;  
s = socket (AF_INET, ...);  
  
int so_reuseaddr = TRUE ;  
  
// SO_REUSEADDR option: use it before binding  
z = setsockopt ( s, SOL_SOCKET, SO_REUSEADDR,  
                &so_reuseaddr, sizeof so_reuseaddr);  
  
z = bind (s, ...);
```

Handwritten notes in Korean (red and green ink):

- SO_REUSEADDR 옵션을 사용하기 전에 설정해야 합니다.
- SO_REUSEADDR 옵션은 bind() 호출 전에 설정해야 합니다.



Socket Option Examples: SO_REUSEADDR (4/4)

- ❖ Conditions for using SO_REUSEADDR
 - no other socket with the same IP address and port can be in a listen mode
 - All sockets with the same IP address and port number must have the SO_REUSEADDR option set to TRUE
- ❖ TIME_WAIT problem can be solved by SO_REUSEADDR
 - see SO_LINGER

UDP BROADCAST SOCKET PROGRAMMING



Unicast, Broadcast and Multicast

❖ Three different address types

■ **Unicast** – class A, B, and C addresses

- point-to-point (I:I) connection
- bi-directional between two unicast hosts

↑ Up to 255 unicast

■ **Broadcast** – host part has all '1's

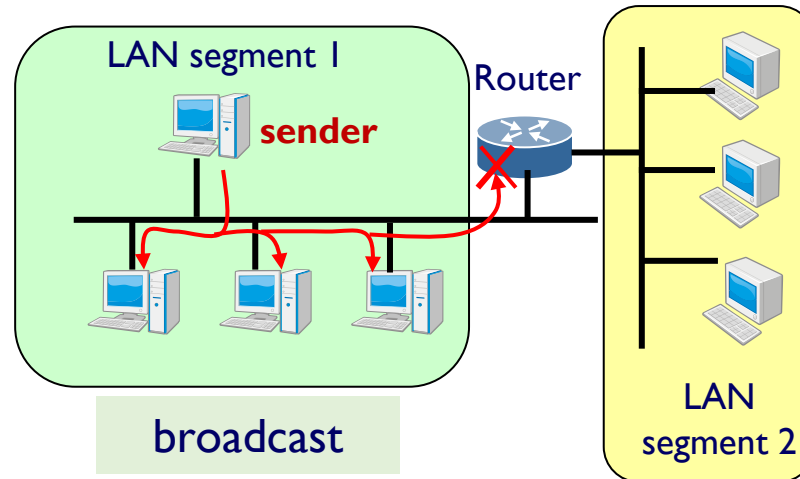
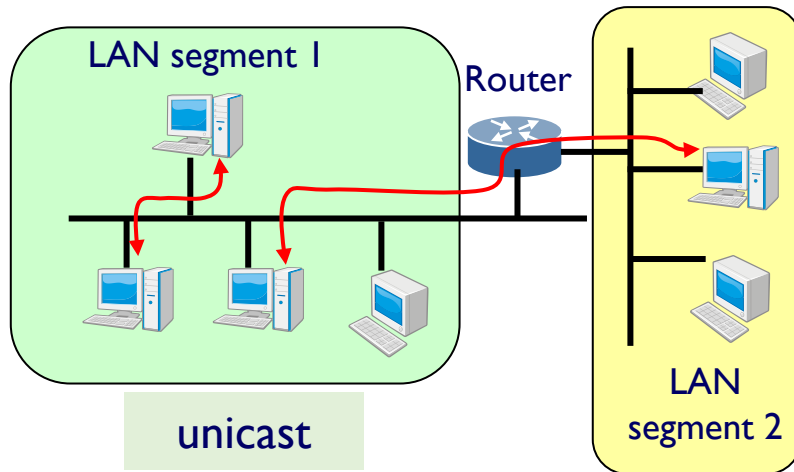
- ex. 255.255.255.255
- point-to-multipoint (I:N) unidirectional connection
 - broadcast messages are sent to all hosts within a LAN segment simultaneously,
 - but blocked at the LAN router

■ **Multicast** – class D addresses

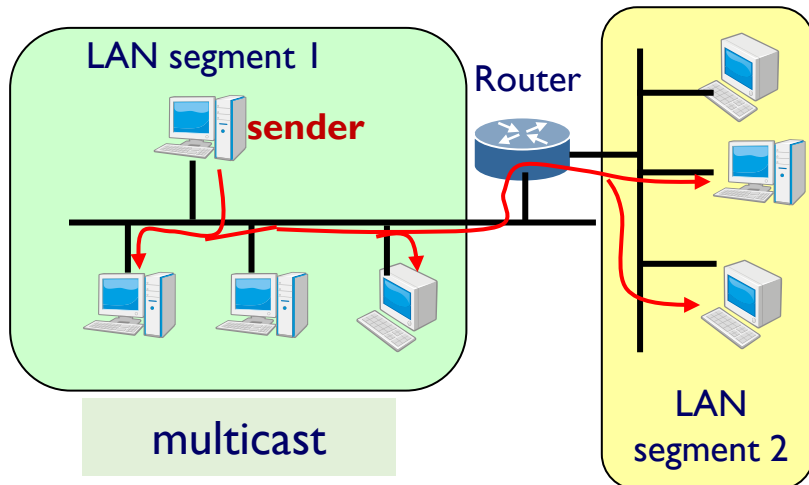
- 224.0.0.0 ~ 239.255.255.255
- one-to-many (I:N) unidirectional connection for group communications
 - multicast messages from any member of the multicast group are received by all other members (hosts) with same multicast address
 - multicast messages can be delivered via routers
- hosts in a multicast group have same multicast address

Unicast, Broadcast and Multicast

❖ Three different address types



- all hosts in the same LAN segment receive broadcast messages
- broadcast messages are blocked at routers

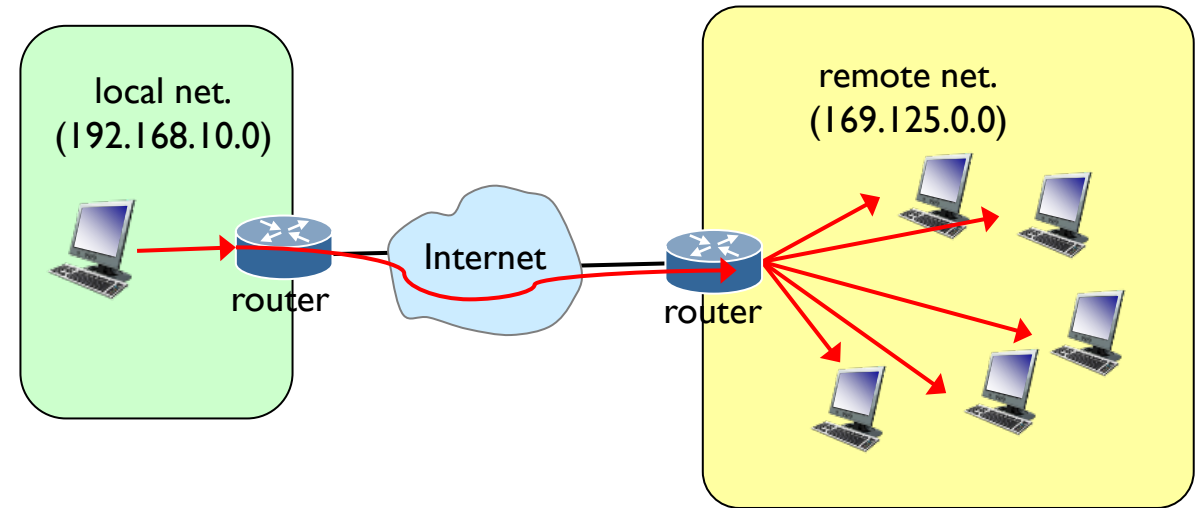


- multicast messages are delivered via routers
- all hosts with same multicast address receive multicast messages sent from any member
- hosts within same LAN segment but not member of the multicast group can not receive multicast messages

Broadcast

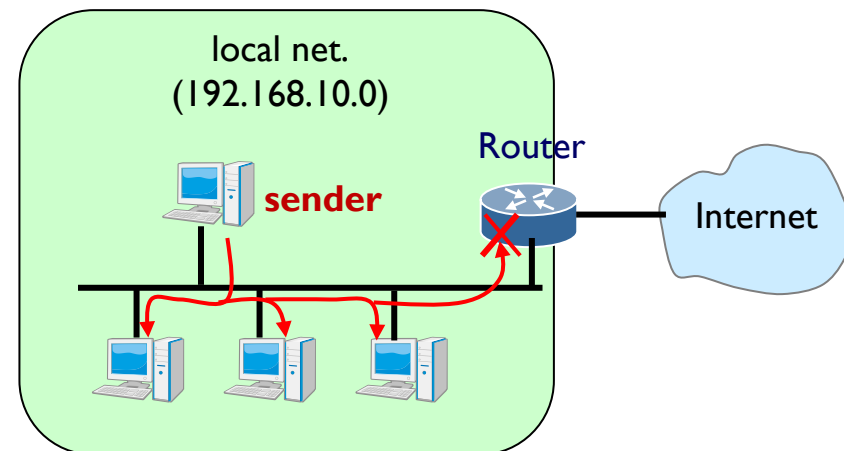
❖ Directed Broadcast *21? 21? 21? x*

- Broadcast messages are sent to all hosts at a target network
- example
 - network id = 169.125.0.0
 - netmask = 255.255.0.0
 - its directed broadcast address is 169.125.255.255
- not support in current routers

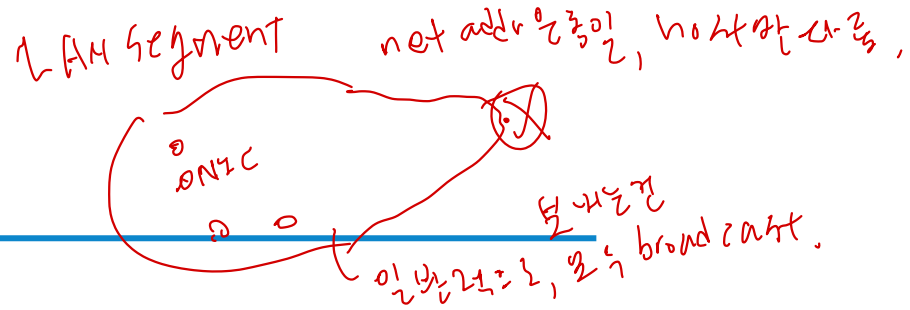


❖ Local Broadcast

- all 32-bits are set to 1
 - 255.255.255.255
- broadcast messages are
 - sent to all hosts on the same physical network
 - discarded at local router



SO_BROADCAST



❖ SO_BROADCAST option

- it enables permission to transmit broadcast messages
- example:

```
int z, s, so_broadcast = TRUE;

/* Create a UDP socket to use */
s = socket (AF_INET, SOCK_DGRAM, 0);

/* Allow broadcast */
z = setsockopt ( s, SOL_SOCKET, SO_BROADCAST,
                &so_broadcast, sizeof so_broadcast) ;

/* Send broadcast message */
z = send(s, ... ) ;
```

Bcast Sender(Client) – Sending Bcast Message

```
main( ) {
    int                s, z, msglen;
    struct sockaddr_in bcastAddr;
    char                *bcastIP="255.255.255.255";
    char                *bcastMsg="This is broadcast message !!";
    int                so_broadcast = TRUE;

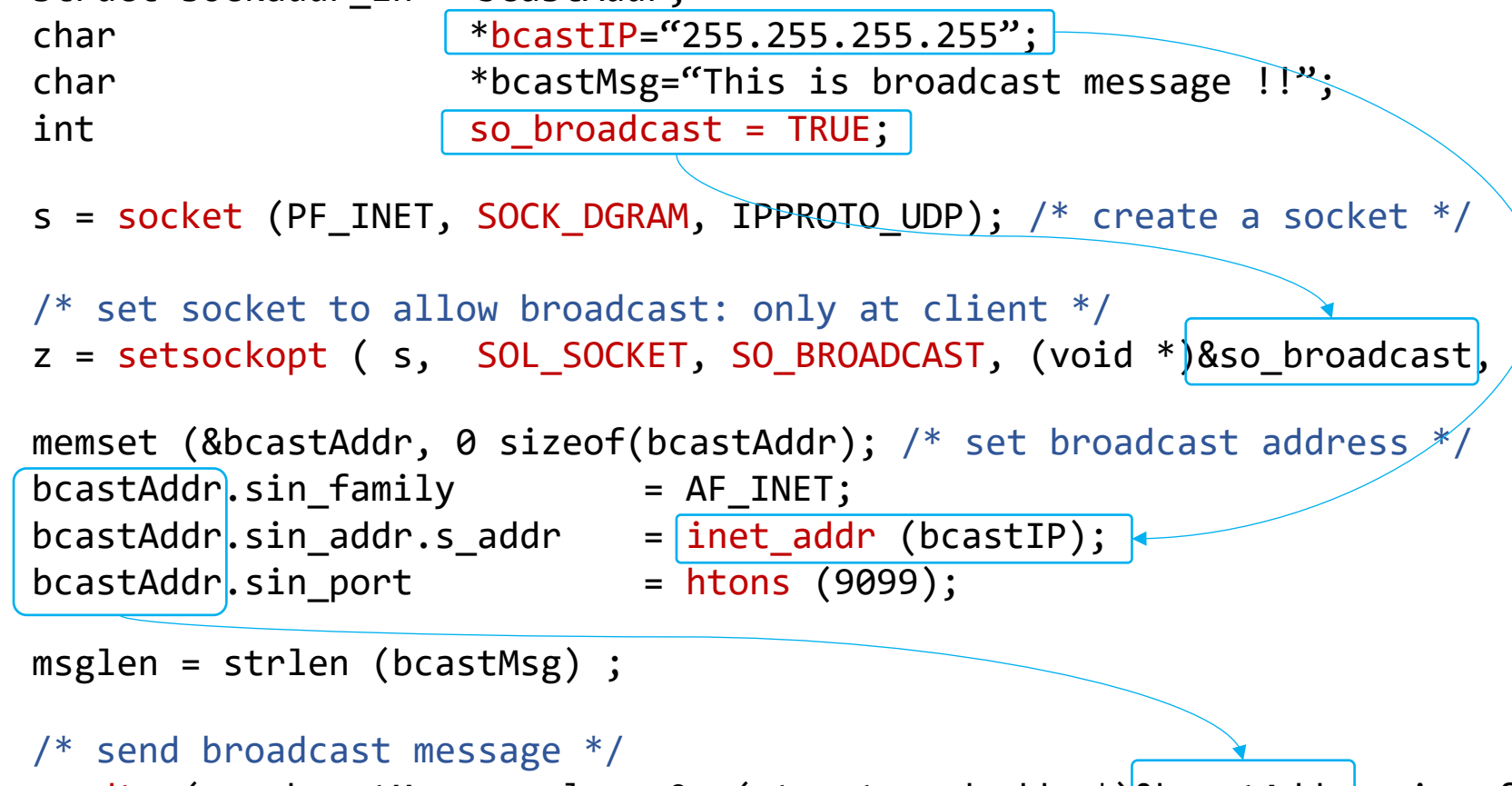
    s = socket (PF_INET, SOCK_DGRAM, IPPROTO_UDP); /* create a socket */

    /* set socket to allow broadcast: only at client */
    z = setsockopt ( s, SOL_SOCKET, SO_BROADCAST, (void *)&so_broadcast, sizeof so_broadcast);

    memset (&bcastAddr, 0, sizeof(bcastAddr)); /* set broadcast address */
    bcastAddr.sin_family      = AF_INET;
    bcastAddr.sin_addr.s_addr = inet_addr (bcastIP);
    bcastAddr.sin_port        = htons (9099);

    msglen = strlen (bcastMsg) ;

    /* send broadcast message */
    sendto ( s, bcastMsg, msglen, 0, (struct sockaddr *)&bcastAddr, sizeof(bcastAddr) );
}
```



Bcast Receiver(Server) – Receiving Bcast Message

```
main( ) {
    int                s, z, msglen;
    struct sockaddr_in bcastAddr, adr;
    char               *bcastIP="255.255.255.255";
    char               recvMsg[4096];

    s = socket (AF_INET, SOCK_DGRAM, IPPROTO_UDP);

    memset(&bcastAddr, 0, sizeof(bcastAddr));
    bcastAddr.sin_family      = AF_INET;
    bcastAddr.sin_addr.s_addr = htonl (INADDR_ANY) ;
    bcastAddr.sin_port       = htons (9099);

    z = bind ( s, (struct sockaddr *)&bcastAddr, sizeof bcastAddr);

    while ( 1 ) {
        /* receive broadcast datagram */
        z = recvfrom ( s, recvMsg, sizeof recvMsg, 0, (struct sockaddr *)&adr, &adr);

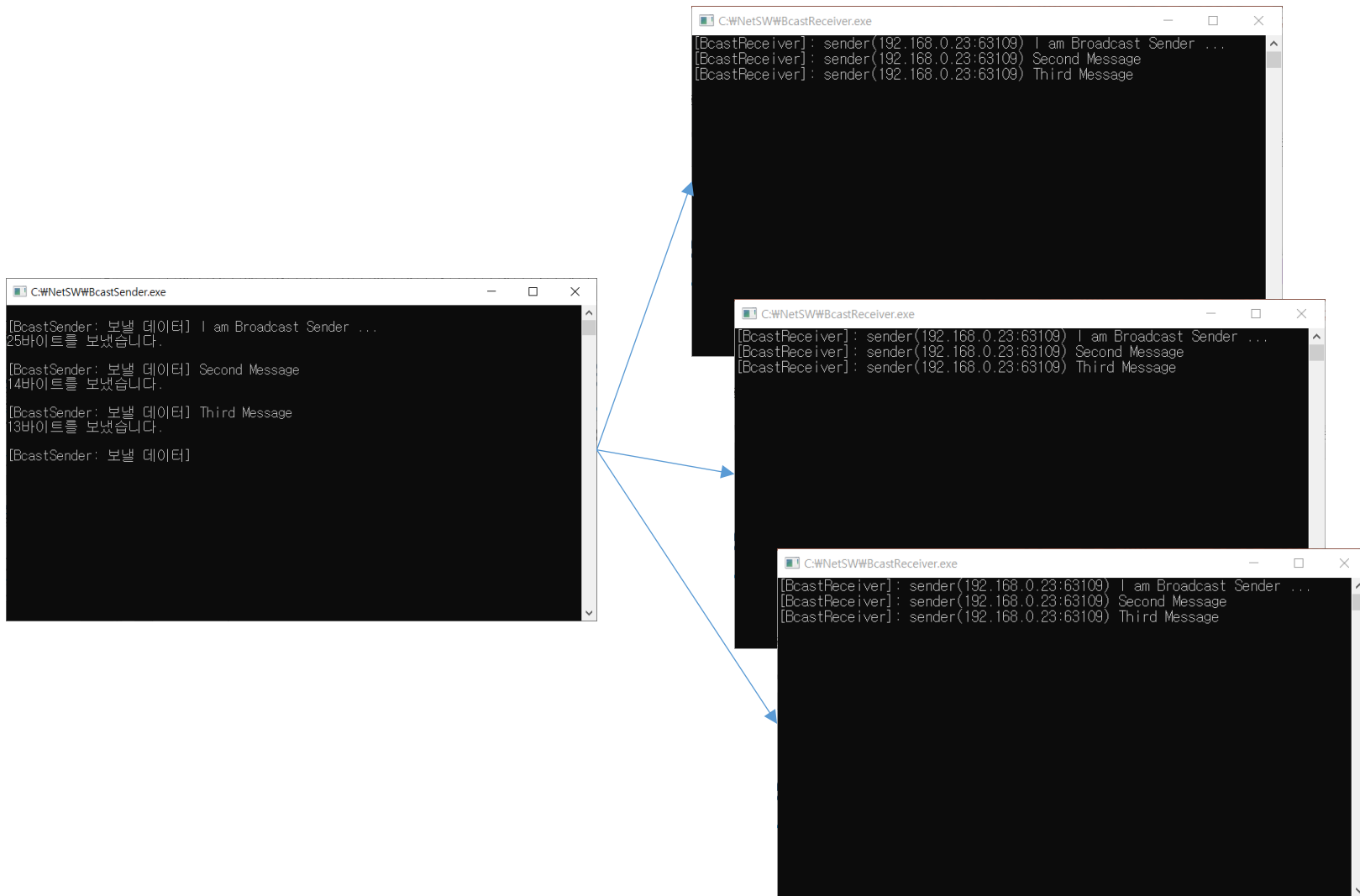
        -- routine to react on receiving broadcast message
    }
}
```

←----- no need of setsockopt() at server

----- Bcast sender address

<http://mmcn.ajou.ac.kr>

Bcast Sender-Receiver Operation Example



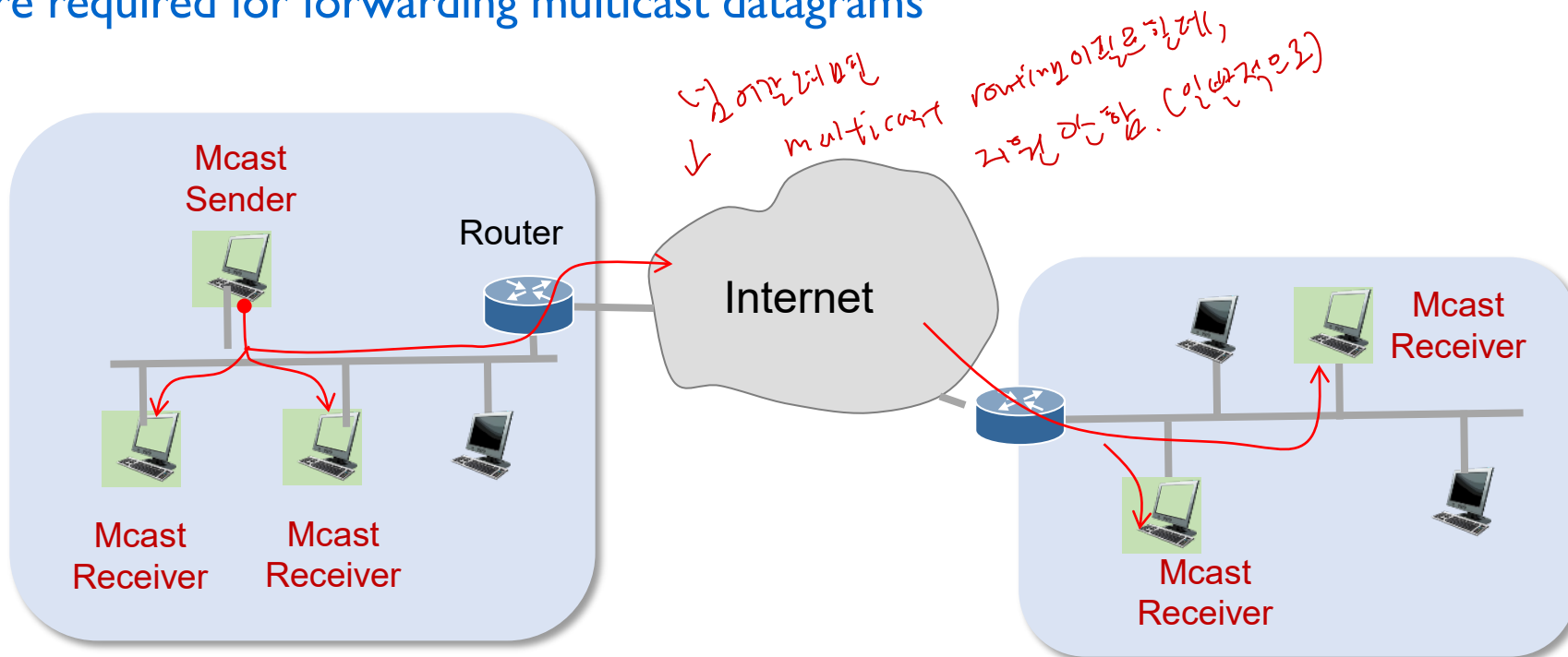
UDP MULTICAST SOCKET PROGRAMMING



IP Multicast

❖ IP multicast

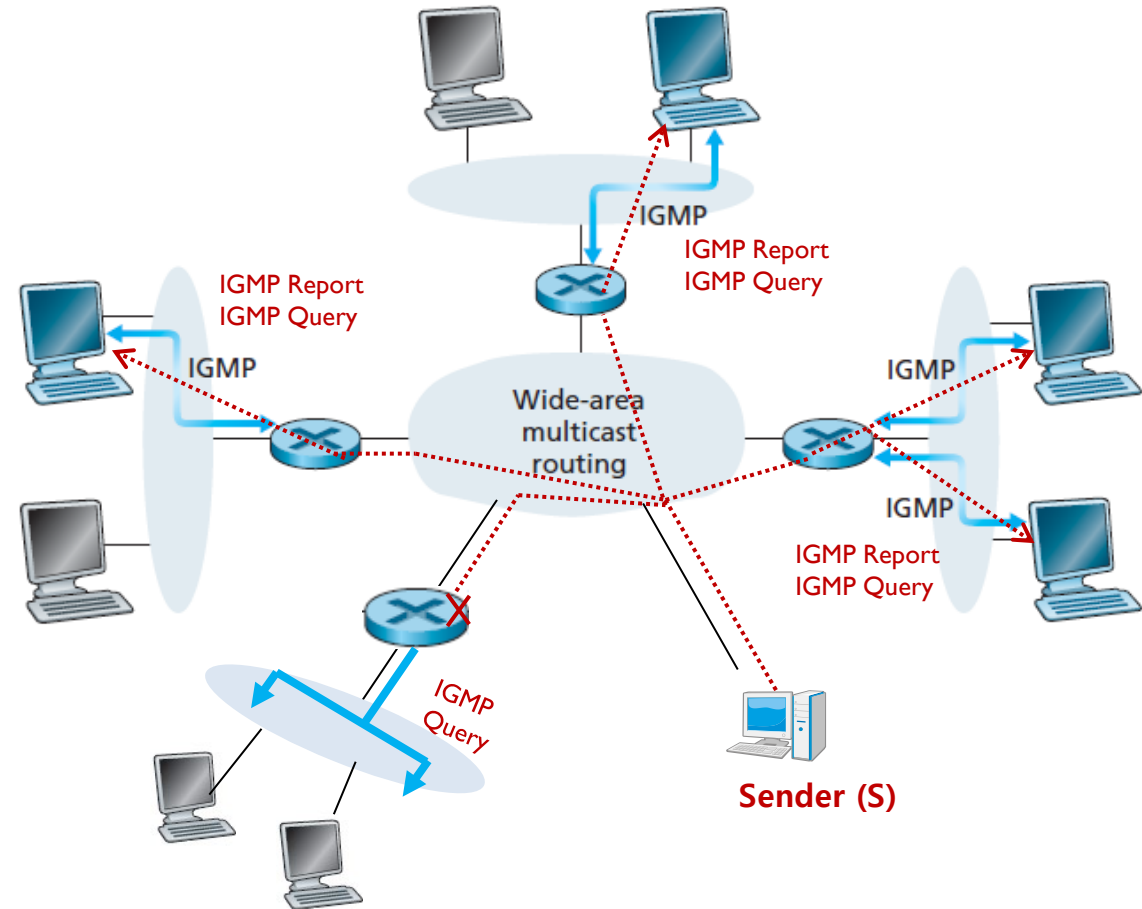
- It allows transmission of an IP datagram to a set of hosts that form a single multicast group
- special multicast routers
 - are required for forwarding multicast datagrams



IP Multicast

❖ Protocols for IP Multicast Application

- IGMP
 - at edge
- Multicast routing protocol
 - in core IP network



IGMP for IP Multicast

(Router 에 2)
NIC에 group address를
안지정하면 안됨.

❖ IGMP (Internet Group Management Protocol)

- located at network (IP) layer, and used by
 - IP hosts to register their dynamic multicast group membership
 - connected routers to discover these group members.
- IGMP versions
 - IGMP version 1 - RFC 1112, 1989
 - IGMP version 2 – RFC 2236, 1997
 - IGMP version 3 – RFC 3376, 2002

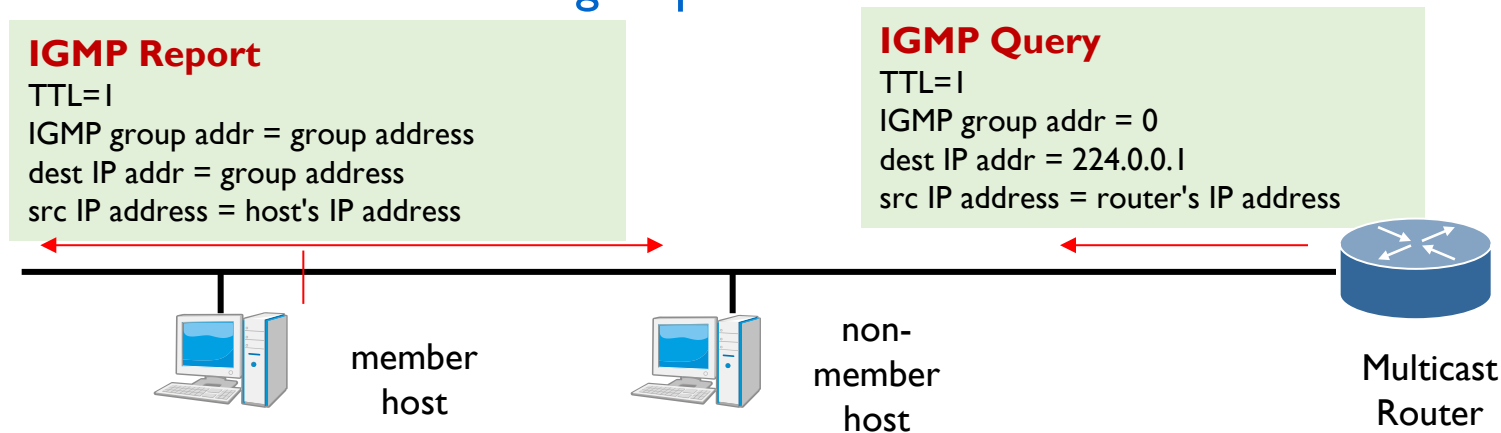
❖ IGMP Message Types

- Membership Queries (general and group-specific)
- Membership Reports
- Leave Group messages

IGMP Operation

❖ IGMP Basic Operation

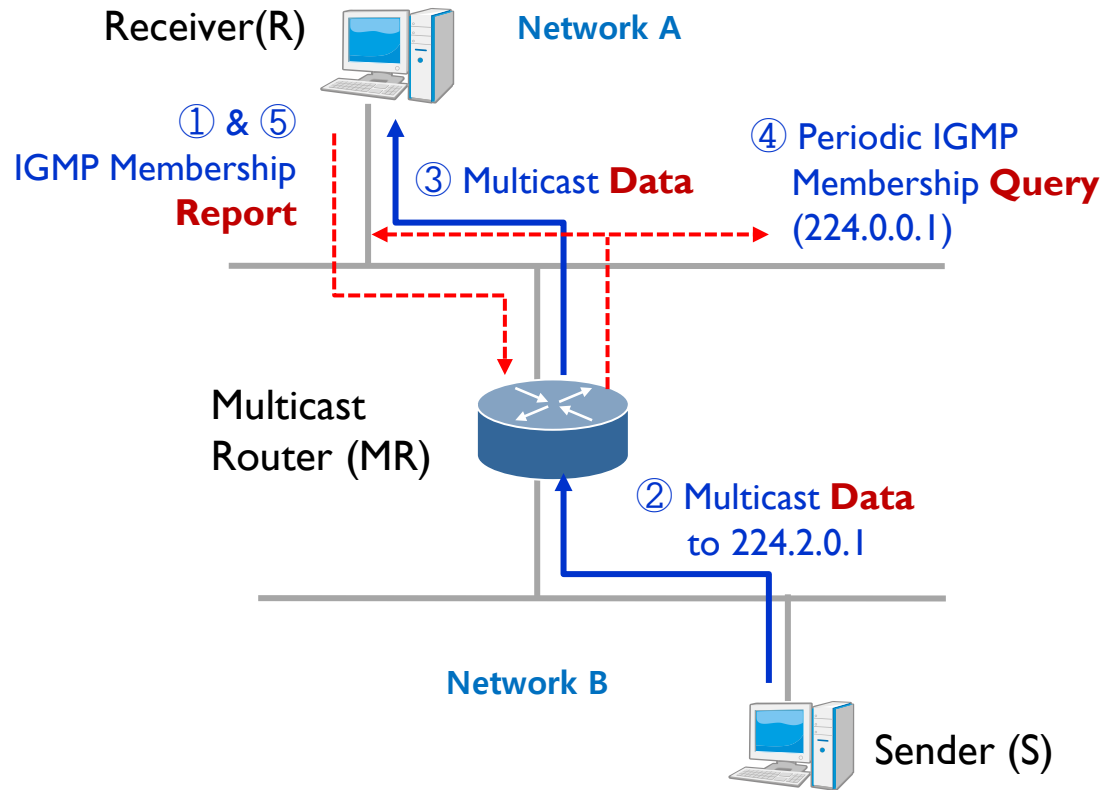
- Phase 1 (Registration From A Host To Router)
 - when a host joins a new multicast group,
 - it sends an IGMP report to the “all hosts” multicast address declaring its membership
 - local multicast routers receive the message and establish necessary routing
- Phase 2 (Polling By Router To Hosts)
 - because membership is dynamic,
 - local multicast routers periodically poll hosts on the local network to determine which hosts remain members of which groups



IGMP Operation

❖ Joining a Group

- Example : Receiver R joins to Group 224.2.0.1



- ① R sends IGMP Membership-Report of 224.2.0.1 to MR
- ② MR receives multicast data from S.
- ③ if any member for 224.2.0.1 exists in network A, MR forwards the multicast packets to Network A, then Receiver receives the packets
- ④ MR periodically sends IGMP Membership-Query to 224.0.0.1 (ALL-SYSTEMS.MCAST.NET)
- ⑤ R answers IGMP Membership-Report of 224.2.0.1

IP Multicast Address

❖ IP multicast Address



- range of IP multicast address
 - 224.0.0.0 ~ 239.255.255.255
 - special address
 - 224.0.0.0 : reserved
 - 224.0.0.1 : all hosts and routers participating in IP multicast
- IP multicast addresses can be used only as destination addresses
 - no ICMP error messages can be generated about multicast datagrams

Multicast Options

❖ Multicast Options

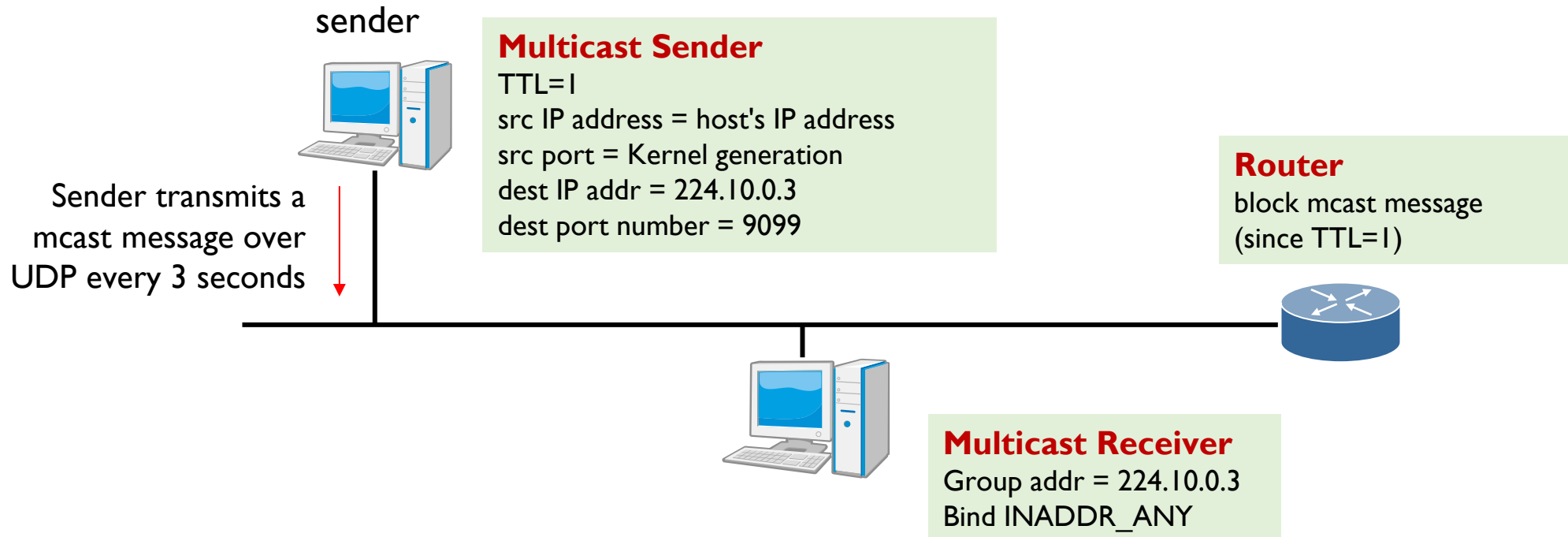
- IP_MULTICAST_TTL
 - it defines the scope of multicast packet delivery
- IP_MULTICAST_LOOP
 - it enables the user to receive sent message from him/her.
- IP_ADD_MEMBERSHIP
- IP_DROP_MEMBERSHIP
 - IP_ADD_MEMBERSHIP and IP_DROP_MEMBERSHIP require the `ip_mreq` structure

```
struct ip_mreq {  
    struct in_addr imr_interface.s_addr; // IP multicast address  
    struct in_addr imr_multiaddr.s_addr; // local IP address interface  
}
```

Multicast Programming Example

❖ Example

- multicast group address: 224.10.0.3
- port number : 9099



Multicast Sender (Client) – sending mcast msg.

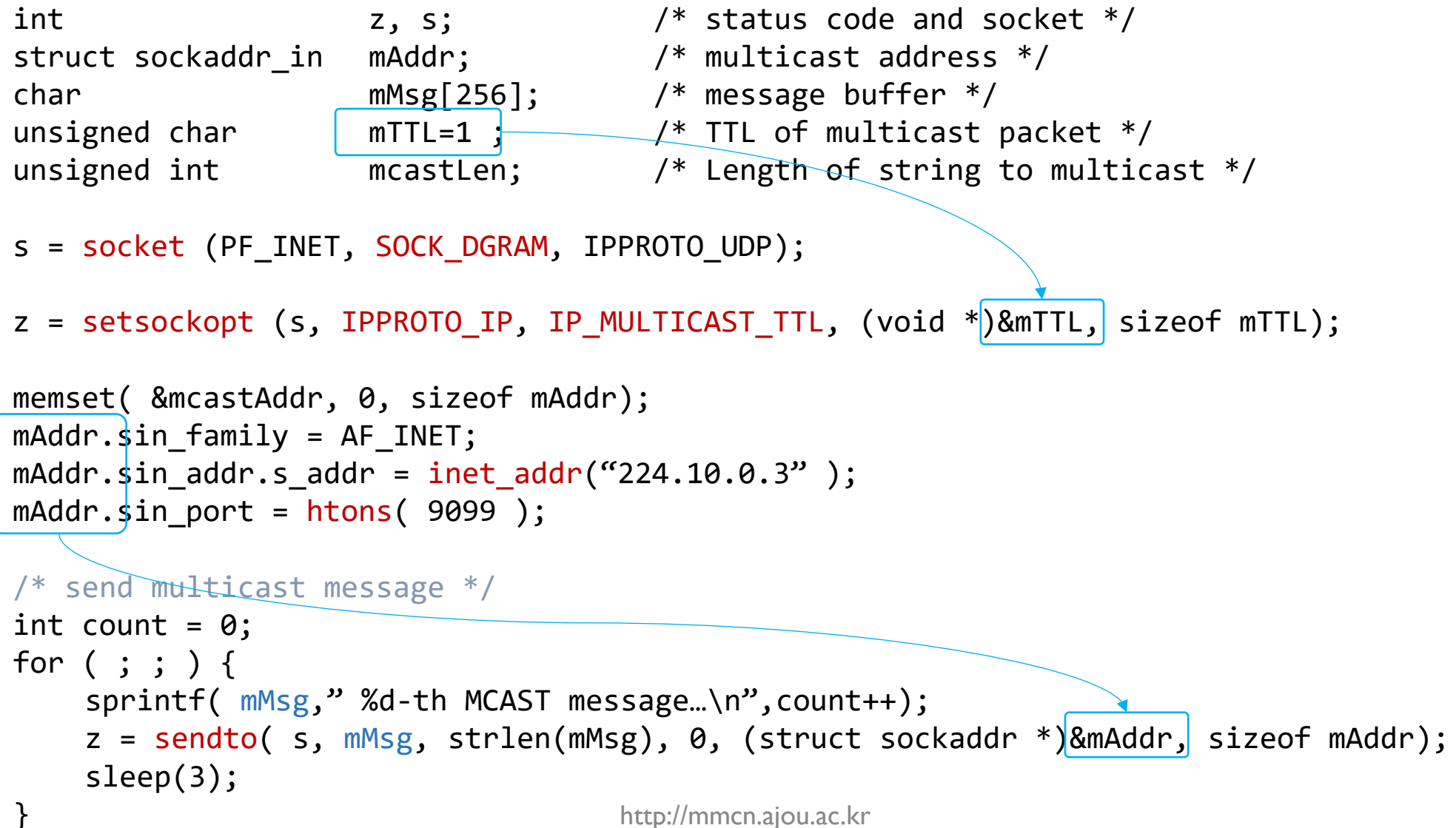
```
int          z, s;          /* status code and socket */
struct sockaddr_in mAddr;    /* multicast address */
char         mMsg[256];      /* message buffer */
unsigned char mTTL=1;        /* TTL of multicast packet */
unsigned int  mcastLen;      /* Length of string to multicast */

s = socket (PF_INET, SOCK_DGRAM, IPPROTO_UDP);

z = setsockopt (s, IPPROTO_IP, IP_MULTICAST_TTL, (void *)&mTTL, sizeof mTTL);

memset( &mcastAddr, 0, sizeof mAddr);
mAddr.sin_family = AF_INET;
mAddr.sin_addr.s_addr = inet_addr("224.10.0.3");
mAddr.sin_port = htons( 9099 );

/* send multicast message */
int count = 0;
for ( ; ; ) {
    sprintf( mMsg, " %d-th MCAST message...\n", count++);
    z = sendto( s, mMsg, strlen(mMsg), 0, (struct sockaddr *)&mAddr, sizeof mAddr);
    sleep(3);
}
```



Multicast Receiver (Server) – receiving mcast msg.

```
int          z, s;                                /* status code and socket */
struct sockaddr_in mcastAddr;                     /* multicast address */
char         mcastMsg[4096];                      /* Buffer of received string */
struct ip_mreq mRequest;                         /* Mcast join structure */

s = socket (PF_INET, SOCK_DGRAM, IPPROTO_UDP);
memset( &mcastAddr, 0, sizeof mcastAddr);
mcastAddr.sin_family      = AF_INET;
mcastAddr.sin_addr.s_addr = htonl (INADDR_ANY);
mcastAddr.sin_port        = htons ( 9099 );

z = bind (s, (struct sockaddr *)&mcastAddr, sizeof mcastAddr);

/* specify the multicast group */
mcastRequest.imr_multiaddr.s_addr = inet_addr ("224.10.0.3");
mcastRequest.imr_interface.s_addr = htonl (INADDR_ANY);

/* join the multicast group */
z = setsockopt( s, IPPROTO_IP, IP_ADD_MEMBERSHIP, (void *) &mRequest, sizeof mRequest );

for ( ; ; ) /* receive message */
    z = recvfrom ( s, mcastMsg, sizeof mcastMsg, 0, (struct sockaddr *)&adr, &adr);
```

----- mcast sender's addr.



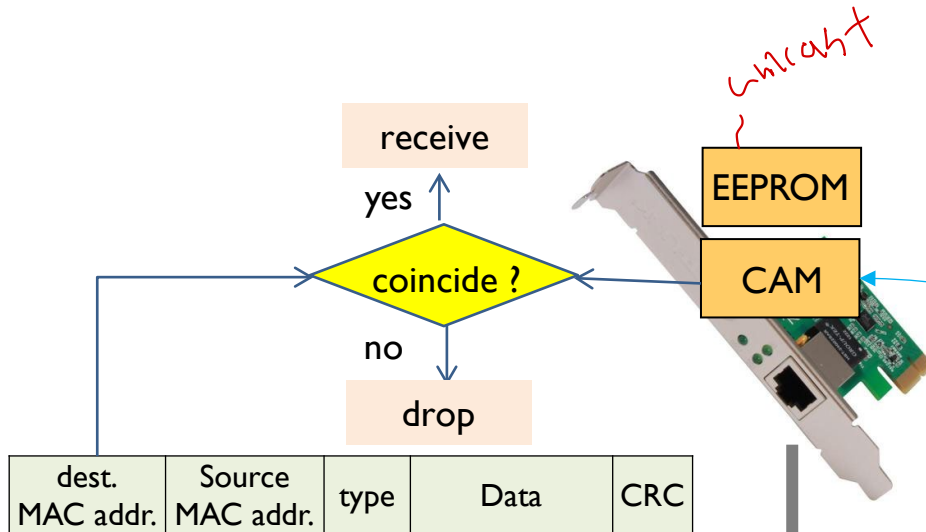
Multicast Receiver (Server) – cont'd

```
/* leave the multicast group */  
z = setsockopt(sock, IPPROTO_IP, IP_DROP_MEMBERSHIP, (char*)&mReqeust, sizeof mRequest);  
  
/* close socket */  
close(sock);
```

Register Multicast Address to NIC

- ❖ NIC accepts frames with their destination addresses
 - its MAC address – stored in EEPROM
 - multicast addresses – stored in Content Addressable Memory(CAM), and
 - broadcast address

```
...  
mcastRequest.imr_multiaddr.s_addr = inet_addr ("224.10.0.3");  
mcastRequest.imr_interface.s_addr = htonl (INADDR_ANY);  
  
/* join the multicast group */  
z = setsockopt( s, IPPROTO_IP, IP_ADD_MEMBERSHIP,  
               (void *) &mRequest, sizeof mRequest);  
...
```



It calls NIC device driver

NIC device driver registers the mcast address ("224.10.0.3") to CAM

Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN



Mobile **Multimedia Convergence** Network Lab.

Homepage: <http://mmcn.ajou.ac.kr>

facebook: <http://www.facebook.com/#!/groups/190702010947314>

